CLAUDE PLATFORM ONBOARDING · MANAGED AGENTS 201

From your loop to theirs

Managed Agents runs the agent loop, the sandbox, and the conversation history for you. Your code only answers when the agent needs something from your systems.

You've written this loop

```
messages = [user(task)]
while True:
    r = client.messages.create(tools=TOOLS, messages=messages)
    messages.append(assistant(r.content))
    if r.stop_reason ≠ "tool_use":
        break
    messages.append(user(run_tools(r.content)))
```

The history lives in your list

You append every turn and resend all of it on every call.

The tools run on your machine

You execute each call and feed the result back.

What comes off your plate

YOUR SIDE

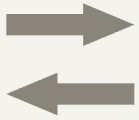
Your code

Sends the task. Answers custom tool calls.

Your systems

The CRM, and anything else only you can reach.

messages, tool results



events, tool calls

ANTHROPIC'S SIDE

The loop

Calls the model and decides when to go again.

The sandbox

A container where the agent reads, writes, and runs code.

The event log

Every event is stored, so a session remembers.

Your code keeps one job: answering when the agent needs something only your systems have.

When to reach for it

Question	Messages API	Agent SDK	Managed Agents
Who runs the loop?	You	The SDK, in your process	Anthropic
Where do tools run?	Your code	Your process	Anthropic's sandbox, except custom tools
Where does state live?	Your list	Your filesystem	A hosted event log

CHECK THESE FIRST

Not currently eligible for Zero Data Retention or HIPAA BAA coverage. Need local files or a private network? Look at the Agent SDK or a self-hosted environment.

Agent, environment, session

CREATE ONCE

Agent

The configuration: model, system prompt, tools.

CREATE ONCE

Environment

The container template: packages and network access. None of your data.

UPLOAD ONCE

Your files

Whatever the agent needs to read: tickets, code, documents.

ONE PER JOB

Session

Runs the agent in a fresh container from the environment, with your files mounted.

events

YOUR SIDE

Your app

Sends messages. Answers tool calls.

Files attach to the session, not the environment.

Custom tools are a round trip

1. The agent calls your tool	`agent.custom_tool_use`
⬇	
2. The session pauses	`session.status_idle, stop_reason: requires_action`
⬇	
3. Your code answers	`user.custom_tool_result`
⬇	
4. The session resumes	The agent reads your result and carries on.

Idle doesn't mean done. Check the stop reason.

LIVE DEMO

A support-ticket triage agent

1. Run the triage

Built-in tools run in the sandbox. Each customer lookup comes back to the notebook.

2. Read the ranking

A "small thing" ticket rises to the top on account data only our code had.

3. Follow up

"Draft replies for the top 3." No new lookups: the session already knows.

What you get by default

From the agent record the API returns. We never set any of these.

`version: 1`

Every agent carries a version number.

`effort: medium`

The model's effort level, set on the agent.

`always_allow`

The permission policy on built-in tools. They run without asking.

Inside the sandbox, the agent acts freely by default. Constraining it is your job.

YOUR TURN · ABOUT 55 MINUTES

Write the part the demo hid

Same agent, same tickets. In the lab, you write the code that answers the agent when it asks for account data.

0 · Preflight | 1 · Set up | 2 · Watch it stop | **3 · Answer the agent** | 4 · Verify | 5 · Follow up | 6 · Clean up

Step 2 stops on purpose. Read what it tells you before you fix anything.